

Hi everyone, and welcome back to the week 3 of Human AI Interaction, where we'll be talking about the interface and interaction patterns for human AI systems.

So what this means is we'll be talking about these little heuristics or rules of thumb, or you could say Lego brick approaches that people have found useful as their designing systems, these little structures that you can reuse and apply across many different places.

So, where did we end up last time? We have been talking about human cognition.

We've been talking about the fact that human cognition doesn't change as quickly as technology, so that's quite helpful because we can start building theories, and we can have a better sense of how people might react, regardless of whether you're showing them something on a desktop or on a smartphone or as a little AI machine in their ear.

Something about the cognition will remain the same.

So we've talked about visual perception and memory, the really important concept of mental models, which we'll be talking about more and more and more, as well as theory of distributed cognition. So

those sort of ways in which your cognition or perhaps even your mental models of how something works get distributed across into the environment. And finally, we've talked about how because we bring some

mental models into interactions and we've mostly interacted with humans or other inanimate objects, often people might behave as if computers were social actors.

So, what does this mean?

So, it means that today we'll be trying to combine the definitions and the big ideas that we've had in week one together with these more theory-based ideas from week two.

So we'll be talking about, well, what does it mean to think about interaction design and interaction design goals, how do they kind of flow out of the heuristics or, sorry, the cognitive models that we've talked about before, and how should we approach it as designers.

So another way of thinking about this is that the interface and interaction patterns and the approaches around these that we'll be talking about today are a way of having a more focused or structured approach of how we can move from the image on the left to the one on the right. So, both having some structure to identify in which ways is the left one really bad, these will be usability goals and heuristics, but then also to think about how can we address the issues we've identified to make that design better, and these will be the patterns as examples of design solutions that have been used elsewhere before and so that you don't have to really reinvent the

wheel every single time.

So essentially this lecture should help us move from designing for interaction overall model and the ways in which to think about it, to some practical examples of how should we or would we build systems that are easy to learn, effective to use and provide enjoyable user experience.

So what we'll start talking about is what makes a system usable and what are the six aspects of interactive systems, regardless of whether they're based on an elevated DAW or a generative AI

Timestamp 3 minutes agent tool, what are the aspects that these systems need to strive for. Then we'll move to talking about the heuristics of how we can achieve this.

So if the six usability goals describe the properties of the finalized systems, the heuristics give you some initial tools of how we can get to those goals. And finally, we'll then delve a little bit deeper into how these patterns and usability characteristics apply to this new generative AI world. What we can use, what we need to think about in new, and what is different.

So, let's start with focusing on the usability goals.

So I'm conscious that these six goals might feel a bit like, duh, of course systems are supposed to do this. But let's go through these one by one on simple examples

and show that there might be perhaps a bit more nuance to what each of these terms means in this particular context.

In particular, what I hope you will see as well is that often the most useful thing to do in design is to ask Timestamp 4 minutes good questions and knowing what questions to ask. So

instead of trying to define how each of these characteristics looks like for any design context you could imagine, so what would it mean to build a system that's safe for use, what are all the things they need to do, we might be defining them essentially through the questions you will ask yourself as you're trying to understand or improve that interactive system design.

So the first one is probably the most obvious one.

For something to be usable, you need to be able to accomplish what your goal is. So if your messaging tool is unable to send or receive messages, it's probably not particularly usable.

In other words, effective means that I can, in the end, get done what I wanted to get done. So even this silly way of representing parking information is technically effective. If I stand there for long enough, it's possible to understand when, where, and how I'm supposed to work or not.

It might be hard, but technically it's possible.

So the key questions here

are around whether or not the product is capable of allowing people to learn or carry out their work efficiently, access the information they need, buy the goods that they want, convey the intent that they'd like to convey for the AI, and so forth. So it's just about, is there a possibility to accomplish this task?

Notice the difference here between effective, as in being able to deliver in principle, and efficient, which means that you're able to carry out the tasks necessary with as limited energy required as possible.

In that sense, having an instant messaging tool that would only allow you to send your message one letter at a time is technically effective, but it's certainly not efficient.

So, the key questions here are around, once people might have learned how your system works, can they sustain a high level of productivity or can they sustain a high level of a good experience, feeling happy, feeling that connected to their friends, whatever it is that your system attempts to do.

so to use this example again it's clearly or it will likely be much more effective for people to understand when and how can they park it doesn't really matter how many times

you've approached or experienced a terrible mess of symbols you would always have to think about it quite hard whereas this might provide you with a much simpler, cognitively much simpler task

In some way, it's distributing the cognition out into the environment.

So we've talked about the first two goals.

Is it effective and is it efficient?

Now we're going to talk about the safe-to-use one.

So really, this is quite obvious. It's about what are the range of errors that are possible if you're using the product, and what measures are there to permit users to recover easily from these errors.

So, if you think about why this keyboard is there, one of the reasons might be that, for example, the delete key is really close to an enter key.

So what might happen very often is that you would accidentally press the delete as you're trying to press enter, which might mean that there's a range of errors that are introduced by just the way of how the keyboard has been designed that might not be present in other places, and that might or might not be easy to recover from.

Making things safe to use is also one of the reasons why, even in very, very old Windows,

you might get these
kind of notifications.

They're saying, has your intent
really been to
delete this record?

Are you really
trying to do this,
just in case this
was an accident?

Because those
might be the cases
where you're not
able to recover.
might just be gone forever.

So that allows
us to move to the
Timestamp 8 minutes next goal, which
is a good utility.

Now utility clearly
depends both on the task at
hand, so what are you
trying to do, and who is it
who's doing it. So for
example, a Unix terminal might
provide many functionalities
and is exceedingly
useful perhaps to many of
you. However, there are
also many people in the
general population for whom
a Unix terminal has
absolutely no utility at all,
because they wouldn't know
how to engage with it.

It certainly does not
enable them to carry out the
tasks in the way that they
would want to do them.

A similar example is
this photo from our first
week, where what's
happening here is clearly
that the utility of the
big TV remote was not
great for those people
who wanted to use them.

It was not the
appropriate sort of
functions because there
were just too many

functions and the users got confused.

So this is another example of both identifying an issue with utility and then a very interesting designerly way to fix them.

So the next to last one is how easy is it for the interface or the system to be used? So is it possible for the user to work out how to use the product and trying it out. Will they mess something out?

How hard will it be for them to learn a whole set of functions? Now, the reason why I'm including this kind of very traditional now chat-based interface is that on some level, this is really easy to use, but on some other levels, it might not be.

So, it's easy because it's relatively easy to start.

You know, where should we begin? I start conversation as if this was a human, so think back to week two.

But what's perhaps hard is that the interface doesn't really allow people to explore the whole range of options and activities or actions that are available to them, which is something that many of the companies are trying to explore right now.

Contrast that, for example, to the apps that you might have on your phone, and I hope that it's not bejeweled. But let's imagine it is.

Likely, these are much

easier and straightforward to learn, mostly because they're often much more specific and focused activities than the highly multipurpose tools, such as the terminal or the chatbot interface, which we don't really know how to deal with, and so on.

So finally, memorability.

So when you read the key question for this usability goal, your brain should immediately start harking back to the last week's content. Think around memory, such as recall is always easier than having to create an idea from scratch, as well as distributed cognition, where you're manipulating your environment to leave traces of your actions, or perhaps what actions are possible, or perhaps where you are in that process and what you could do next.

So again, memorability is about the interface and the ways in which you enable and help users to remember what they should do to carry out tasks, and especially perhaps so for the things that they do infrequently, where the recall is not just based on them remembering they're supposed to do this task, but it's about, well,

I'm supposed to do it, how was that happening again?

So overall, you should think of these six goals as the first, the, line of defense, one that you can always take when you're analyzing an interface or you're

trying to develop a system yourself. So what you should be thinking is along the lines of, well, what are the user's goals here? What mental models do they bring? How are these aligned with the conceptual models behind the scenes of how the system works? What would it mean to make things efficient to use? What are they trying to do and what is the energy here? Is it time? Is it the effort? Is it the number of people they need to reach out to? What does utility mean to users? How hard will it be for them to learn or remember how to execute tasks? And again, remember the gulf of execution concept from last week and so on.

So overall, though, these tools are still only here to help us identify problems, what could go wrong, but they don't really directly help us figure out how to solve anything, any of these issues that we find.

But before we start talking about solutions, there's one final point I want to make about the relationship between usability, which is what we've talked about, for example, defined for simplicity through six goals, and user experience.

A simple definition of user experience here is that it talks about how the system feels to the user, when they're attracted with it, whether that is holding it, working with it, or having to be

bossed around by it, for example, if you're an Uber driver or another member of the gig economy.

So, one of the major implications here, then, is that you can't really design a user experience.

experience. You can only design for user experience.

The experience is always something that the user gets when they're interacting with your system.

And you can kind of shape how those interactions might look like, but you can't force an experience on them in the same way as you can't force people to enjoy a movie. You can just try and do a movie that a lot of people might likely find enjoyable.

When you start thinking about objectives and start comparing usability and user experience, you can see how usability is much more restrictive, right? So it might be effective, efficient, satisfying whatever goal you want. But user experience is often much broader. It might be talking about aesthetically pleasing, it might be talking about how enjoyable it is, how motivating, rewarding, is it supporting of your creativity or of your goals, and so forth.

So one of the common misrepresentations associated between this usability versus user experience distinction is that design and associated user experience is often driven by how beautiful

something is and those beautiful things just work better in the sense of providing a great user experience, whether that is with sleek apps or cars or furniture or Macs. But the question is, is that really true in general?

So this is often associated with the notion of the role of the designer being to make things pretty or appealing or slick. For example, this little juicy salive, which is essentially a fairly expensive or inexpensive, depending on how you look at it, inexpensive sculpture to put on your work desk in the kitchen.

However, another way of thinking about this, the user experience that's underlying this, is when you don't think about how appealing it is, but you think about the goals and what happens when you try to achieve these goals.

In this case, the juicy satellite might look beautiful, but then make a massive mess all over your kitchen bench, kitchen desktop, and throw pips into your drink, whereas the juicer that costs a fourth of a flat white somewhere, will actually do a much more effective and enjoyable job in the end, because Timestamp 15 minutes the enjoyment is not about looking at it, the enjoyment is about the drink that this produces.

And just in case you were wondering why everyone keeps

on talking about juicy salads, it's partially because Don Norman has put that picture on top of his emotional design as I think a good example of what it means to design beautiful things and why they sometimes work and sometimes don't work particularly well.

So one way of thinking about this is that usability and user experience are at least two dimensions that are relatively independent on each other, and that we could position to this two-by-two matrix.

So what this means is that you might have products that are beautiful and also really usable and that's often where we want to get to. But then there are the other three squares that we can talk about.

The beautiful but unusable one is the juicy cellar.

It looks amazing, it's very appealing, it's a sculpture, it's art, but it doesn't quite work.

The charming but usable is the juicer that actually works, but it doesn't really look very nice and it's not something you'd show proudly to anyone coming to your home.

And obviously, the stuff that nobody wants is the stuff that's not usable, and also doesn't look like anything that you'd like to be around, so let's just not do those.

So, just to wrap up, the idea here is that the usability itself, or the usability goals, aren't the

only thing that matters,
and that user experience
is really important too,
but it's also deeply
tied to the usability of
the system that you have.

Okay, so we've gone through
usability goals, we've
talked about how that
relates to user experience,
so let's move on to
heuristics. Let's move on to
thinking about how could
we design for usability.

Now what this means is
that we're moving to
talking a bit about the
design decisions and
Timestamp 17 minutes choices, which are the
properties of the resulting
system that can lead
to a usable product.

As you'll see, the
heuristics again suggests
questions and associated
design patterns that
if followed make it more
likely that a system
will be understandable
and usable for users.

But before we can get to the
individual actions, let's
have a look at the concept
of heuristic. What it
is, where else are we talking
about heuristics, and so
on. This can hopefully then
help us understand a bit
more about how to use these
heuristics for design.

You might have heard about
heuristics, for example,
in programming or in
similar places. This is
because the term heuristic
itself just refers to a
strategy that is derived from
previous experiences with
similar problems. So what
are the implications?

You can think of these strategies as your best bets or rules of thumbs, something that might just likely work because it worked on similar things before, but might not actually work in your particular instance. So, this is what the second paragraph says. When Timestamp 18 minutes you are applying a heuristic, you rely on information that is easily accessible, but might or might not be directly applicable to your problem.

There is also a second way in which this term heuristic is used, in psychology and neuroscience, where what it means is that heuristics are, again, similarly simple rules that can be applied, but which might have been learned or hard-coded by evolutionary process. And these have been proposed to explain how people make decisions, come to judgments, solve problems, and just generally approach situations where you face incomplete situations or you need to make choices quite quickly.

The original research, which I really recommend you to read if you have a moment.

And what I really want to point out is especially the notion of satisficing as being crucial for design. So what this means is that your users will usually not do a detailed analysis of your

system and really think it through, but will just make choices that seem good enough at the time. Timestamp 19 minutes And often what good enough means for a participant is a much lower bar or user is a much lower bar than you would imagine.

So the way that we will use heuristics in design is based on both of these aspects. The first one more or less talks about how designers think about solutions.

So the heuristics are about the design heuristics of how to perhaps resolve a situation of, say, creating an understandable login flow. Because you've already seen that problem resolved in many other places before.

The second one then talks about the potentials for errors or understanding of the mental models that your users might bring along to your system. So that is perhaps what creates the problems in the first place.

So let's have a look at what this means in practice.

OK, so what we'll do is now step through these Jacob Nielsen's heuristics one at a time. These are all really pretty old, but have remained a useful tool in the designers' toolkit for quite some time.

So what I'll be trying to do is always talk about what that particular heuristic does or what it talks about, and use some examples, both traditional ones, as well as sometimes some from generative AI.

So the first one
seems very obvious,
and it is easy to get it wrong.
When we interact with
physical objects, we mostly
see the effects of our
actions right away. So if
I try opening the door,
press the handle and push,
I expect to see something
happening. It might be
the door opening, or it
might be my realization
that the door is locked,
because it does not open.

Now, this is not
necessarily the same for
digital systems,
where actions often
initiate otherwise
hidden or delay changes.

So the heuristic says that
the system should always
keep users informed about
what's going on, through
appropriate feedback and
within reasonable time.

So imagine if you
were selecting your
interests or hobbies
or whatever this is.

You clicked on things, but
it didn't change color,
it didn't put a little
tick mark. The system will
Timestamp 21 minutes just remember that you
clicked it without any change
on the interface. That
would mean that there's
no visibility, there's no
feedback, the action has
been recorded on the system,
but you have no chance
of knowing whether that
has actually worked or
has not. And that would be
a problem, because then,
as you would imagine,
people get really confused.
We might get similar

instances of signifiers in other places, physical places. So, for example, the old Mac charge cables have had this little light that indicated whether, A, the energy is going.

There was light at all. And whether the Mac is charging, orange, or whether the Mac is fully charged,

The new Macs don't have it. There are benefits to this, because you can just use the same cable for everything, but you also lost this ability of getting this immediate visibility and feedback, and the system needs to deal with it in another way.

We also see this idea of providing visibility and feedback of things that are happening behind the scenes within generative AI systems.

So you might have seen this slide from week one, but the idea of, I have provided some intent to generative AI, and now the field is still trying to figure out how to represent what is actually happening behind the scenes, how to make that kind of visually accessible, and how to provide feedback for whether or not my intent has been communicated well in various ways. This, I think, what Persplexity does is what now has been copied by GDD5, what Claude does as well, all the thinking models do that. So it's perhaps starting to become a design pattern, but it's one that is still

being created. And the purpose of all this is really to engage with the visibility and feedback, to make you aware of what has happened, rather than just sitting there for 10 minutes without even indicating how much time is left.

The second one, again, seems relatively obvious, but has a few catches.

By referring to user language, the first part mainly means that our system should speak and label possible actions using terminology that maps to users' goal of conceptual models or mental models, and how they think about how the technology works, rather than the implementation of that technology. So as a user, you would be surprised if your thermostat labeled actions as reduce valve 5 and 7, as opposed to heat up the living room.

So, one potential way of providing some skeuomorphic designs, so designs that match onto what's happening in the real world, is to draw on users' existing conceptual and mental models that they bring from elsewhere.

So, for example, if you know that your users kind of assume that a digital page should still have highlighting, you can provide design that kind of allows them to operate on that page as if they were holding a highlighter, they're just using their finger instead of it. Or you can provide something

that acts as a compass,
even though that's a thing.

In both cases, it makes
it easier both for the
designer and for the
user to understand which
actions might lead to
which goals and why.

Now, you can see that these
attempts to match real
-world conceptual models can
also be problematic, such as
if it is relying on schemas
of talking to a human,
which then leads to
anthropomorphisation of systems and
can lead users to have
wrong mental and conceptual
models of how the system
works, how and why to interpret
the output and what to do
if that output is wrong.

So the third heuristic is
one about user control and
freedom, and we'll see that
in traditional systems,
that often meant kind of
enabling a user to recover
from an error when something
happened that they didn't
intend or now they realized
that they hadn't intended
it and now they need to
revert that option. So this
is a joyful example of the
situation where the joke is
in assuming that user
control is the active choice
of not responding. So I
could ask, I didn't respond,
it was a choice and therefore
it's okay. whereas our
internal simulations of how
this could happen mean that
this is unlikely the the
most plausible option is
probably because someone has
left they just left their
computer sitting there so
in effect this situation

would offer the illusion of control user has been asked but then do an irreversible action as a result so part of giving your users control and freedom is that you don't want to lock them into decisions from which there is no escape So imagine how much more annoying booking on Airbnb or other places would be if, for example, you couldn't return to your map or to your preselected hostings and whatever you clicked on first, you kind of would have to restart entirely from scratch.

Timestamp 26 minutes So what control and freedom also means is interesting when it comes to generative AI. And as you can see on this video, there's many more degrees of what a control or freedom to interact with whatever the system has created for you could look like when you do interact with Gen.AI objects. And I think, as with everything, there is still a lot that's being developed and designed within the field as we're trying to understand what these tools would actually look like.

So we will try starting to go through these with a bit more speed, as I hope you're starting to get the gist of what these heuristics mean and how you might be able to use them.

So the consistency and standards one should take you back to the cognition lecture last week, from where it should be obvious that it will be

easier for people to engage with your system if the same action is not represented by multiple objects, or if it's generally consistent with how particular objects react and what they mean.

This is particularly important if your users are bringing conceptual models from interactions with other apps or systems or devices.

There are some great examples of how consistency and also many other heuristics have been instantiated, for example, by Apple, into their developer and design guidelines. Well, part of the ideas is that many apps should behave very similarly within that ecosystem, and for common actions, you would have common representations.

Some of the activities that happen later in this lesson will be to really go and engage with both the existing guidelines and heuristics that people have developed for pre-Channel AI systems, but then also talking a little bit about how that changes now that we have GenAI systems and what are the things that the field's trying to develop.

So we'll see that in traditional UX guidelines, these can get really specific.

For example, Apple will enforce that your menu bar menus should typically include the following options

and these options should be in the following order.

So we won't go into details just yet. I just wanted to put this slide here to show how for the human AI interaction guidelines the specifications are much more high level.

It's about you should support a particular type of interaction, so you should match social norms or you should support efficient dismissal, rather than really prescribing the specific ways in which this gets implemented.

So the next heuristic is around error prevention.

So while control allows you to potentially roll back an action that you didn't want to do, it's often much better to prevent users from doing errors in the first place.

So the idea is often that you identify kind of dangerous situations within your app. What could the user want to do and what might they accidentally choose to do instead? If you remember, for example, the Timestamp 29 minutes keyboard, that was an example of kind of physical error prevention gone wrong, where instead of pressing enter, you would delete the particular piece.

So as with everything, Japanese have a beautiful term for this, Pokaryoke, which is a mechanism where by design you prevent people from doing

a mistake that could have been really serious.

For example, if you have a manual car, you're unable to start the motor unless your clutch is fully depressed.

because if you didn't have it depressed it could mean that you start the motor and it immediately tries turning the wheels.

So that would be an example of this mistake proofing or error prevention that we're talking about. You can see many of these instances in digital apps and on your desktop systems as well.

Some of these might be also thinking about what are the immediate actions, right?

So people often will create, kind of accidentally do something and then are tired of notifications so would wave them away so for example that's why you might choose to design the system so that when you're asking whether the question is end the game and lose all progress the primary thing is on no so if you haven't done that if you don't think about it and you just press an entry or something it will not end your game you have to make a choice active choice to to end that game so that's kind of a error prevention right there similarly if an activity or action has some side effects that you weren't aware of such as changing the zoom will restore to your iPhone you might want to make it again a very active explicit choice so that you're sure as a designer that the participants

know or your users know what they're doing and why.

So this heuristic goes hand in hand with heuristic fall, consistency and standards. So how do they actually differ?

The number four, consistency and standards, talks about how the actions that needs to be done are actually done within the system, both within your specific kind of universe of the things that you allow people to do within your app or system or whatever it is that you design, as well as comparing it to all the other systems that are similar and that your user is used to interacting with. So if you build an app, your app needs to be quite similar in many of those mechanisms, consistent with many of the mechanisms that other apps use.

In terms of recognition, you're talking more about once the person is within your universe, within your app, your system, and has learned how to engage with it, how do they know what is currently possible to do. So reducing the gulf of evaluation.

So this might mean kind of providing possibilities that are available. You'll see this as filling of the words or possible variables in, say, a coding system. Or it might be things in games where you're kind of showcasing what is it exactly that the options are or which buttons to press. This might not be really necessary and

people don't pay attention to it for the actions that are quite common. But if there's an action that you don't get to use as often but you know that you want it, again, the recall makes it easier to engage and not lose time.

Next heuristic is about flexibility and efficiency of use. So the idea here is that if people start, they might need something relatively simple and not too complicated and everything needs to be visible and available for recall.

Whereas if people use your app frequently there might be some underlying models or engagements that you want to speed up.

I would encourage you to pause this slide for a little bit and just read how the types of questions that might be asked by beginners, intermediates and experts

Timestamp 33 minutes can happen. Again this is not true for all the apps or systems you might build but that tends to be true for quite a lot of apps and systems. What I think is interesting here is that most of your users, if you're successful, will be intermediates, right? So there are people who've already used your app or used your system, might not have used it very recently, but already know what to do. And then you'll have some light tail of experts. And it is for these experts that you might start providing additional tooling

that will increase their efficiency so you're no longer kind of trying to guide them and build their mental and conceptual models about how your tool works but right now you're really trying to provide them with the most energy efficient way of engaging with your system so this is an example of shortcut stickers that you can place on your keyboard if you're working a lot with adobe illustrator so that it becomes a way of reminding you what those shortcuts are. So the act of having the sticker is not part of the flexibility, it's not part of the heuristic, but the fact that Adobe Illustrator has this hidden kind of layer of tools that allow you to be more efficient, that's what falls within that heuristic.

There's lots to be said that things can be or should be aesthetic and minimalistic. So the idea here is that you're really trying to focus on signal-to-noise ratio.

So what does this mean? In the old age, websites might look like this.

So it's like a jumble of lots of things that it's really hard to figure out what is it you're looking for, where the information is. There's potentially a lot of signal, but each of these signals is kind of hidden among a lot of the noise.

Compare this to, say, a slightly newer web page.

Timestamp 35 minutes Here, you will see, as you will

see on many of the landing apps, everything's optimized to a particular action or a small set of actions. So what is it you would want to do? You could book travel, you could look at flight status, you could do some check-in, and you might think about your trips, and then you have some other smattering of common tasks around it.

So if the previous heuristic has tried to simplify users' cognition by kind of channeling them by increasing the signal-to-noise ratio and highlighting the information that might be important to them, this heuristic is talking about well how do you help people evaluate and get a sense of whether or not something has gone wrong, what the status of the system is, and how could they recover from this.

So as an example, I don't know how many of you still tweet or X or whatever it is right now, but this is an example of showing you how many characters you have. Timestamp 36 minutes left out of your 180. And then if you do go over, it kind of shows you very explicitly what the error was, too many characters, and how many of those characters are actually the ones that are causing the problem.

Similarly, within Gen.ai systems, you start seeing these types of patterns emerging. So very often in chat-based systems, you will have the opportunity to

retry, or you could retry with a different model, with extended thinking in this case. Or very often, some of these systems, like Claude, start allowing you to branch, so you can kind of go back in that conversation because it hasn't gone the way that you want it, and you can branch off a different conversation by either just rerunning, rerolling what LLM gives you, or editing the opportunity.

So this is one way in which people are currently thinking about embedding some of this error correction in something that is essentially a dialogue that That is hard to kind of shift halfway through.

Finally, help and documentation.

So, obviously, it's better if people can just use your system naturally.

But having some documentation and helping people understand if they do get stuck and thinking very carefully about what it means for them to get unstuck is also really crucial.

So what I want to leave you with is this. Although the 10 usability heuristics have been presented as independent, they're all really related to each other and also they're related to the six usability goals that we have talked about before. So what I want you to do now, if you have a little bit of time, is kind

of pause this, go back through the lecture and really try to see which of the heuristics link to which goals. Ideally, you could create little spider webs like these, where you can think about how the heuristics then link up to the individual usability kills.

So over time, many designers have applied these heuristics to create common patterns that solve common engagement problems. These could be in boarding flows or login pages, perhaps things like how do you enable users to filter or sort a larger list of items. So, what I've posted into Keats is this link to a short video that provides an overview of design pattern libraries, so websites that assemble and curate these design patterns.

And these are assembled for pre-gen AI UX technologies.

And what I'd encourage you to look through is not necessarily go in a lot of detail through every single web page, but just at least have a look at the video and spend a couple of minutes, just browsing through at least three of the five websites that they might offer and suggest, just for you to get a sense what does it mean to think about a design pattern and what are the kinds of patterns that you will

Timestamp 39 minutes see there that you'll then recognize within apps.

There's similarly interesting questions

about how these design
pattern libraries
will look like for
Gen.ai systems.

And as I mentioned earlier,
these are far from being
as detailed as the current
UX patterns, which I've
had probably about 20 years
or so to develop, but there
are really some few interesting
websites that try to
curate patterns that seem
interesting or useful or
exciting or perhaps starting
to become common, and
then they kind of bring them
all together into a nice
little place that you can
look at. Shape of AI is one
of these and is linked in
Keats, and I would suggest
that you go and have a look at
it once you have
a little moment.

We're also starting to see
multiple sets of guidelines
that are trying to curate
a similarly focused
set of best practice that
people should think about,
akin to the usability
heuristics by Neeson from
decades ago that we've
talked about earlier. The
blog post that I've posted
at Keats outlines the top
five ones that are
relatively recent, and again,
Timestamp 40 minutes it's part of your reading
materials for this week.

What I want to do in the
rest of this lecture is
just pull out two guidelines
that are five years
apart one of these coming
from microsoft the other one
coming from ibm where
the 2019 one was talking
about human ai interaction

but it was pre-generative ai so it was talking about systems that are making decisions and are kind of deep learning systems where you might have been thinking about the data that one thing whereas the 2024 ones are very recent and they really talk about what what does it mean to design for and with generative ai applications

So, I will not be going through all the guidelines in these papers, as it would be really boring for you, but also as you will get to engage with these in depth during one of the LGTs, where you'll be using the individual guidelines and picking and matching the guidelines to stress test a system that you have developed.

What I wanted to do here, though, is just to really illustrate the format that people took to specify the guidelines. In this case, a simple title, make clear what the system can do, which also indicates that there's one thing that the system does, or a small number of things that the system does, and a tagline to help the user understand what the AI system is capable of doing, which is followed by good and bad examples from existing interactions.

the 2024 Gen AI paper is taking an analogous format, although the specific examples are missing. Not necessarily because the

authors would be lazy or couldn't be bothered to add them, but I think because there might not be enough canonical approaches to include yet. So, in their case, notice also how they're talking about designing for different things, for mental models in this case, and how it's important to communicate how to work effectively with the system, not necessarily what the system can do, but how the human can work with the system effectively, considering the user's background and goals, and then they have some sub-guidance that we can look into during the LGTs.

So, what I tried doing is pull out two interesting comparisons between what seems to be underpinning some of these guidelines.

So, the 2019 ones, the deep learning ones, are really focused on making AI behaviour predictable and consistent.

It's about helping understand the users understand when the AI would act, what it would do, why, and the assumption there is that AI systems should behave deterministically, so that there might be a decision that's being made and that decision should be made repeatedly in the same way or at least following some rules.

So some of the guidelines for example is made clear why the system did what it did.

Whereas the Gen AI systems

tend to, or at least the guidelines that IBM has selected, tend to really embrace the notion of it being a generative variability.

You're generating lots of things rather than getting one particular thing out.

So the guidelines help users manage and leverage the fact that generative AI will Timestamp 43 minutes produce lots of different

things and that that might actually be a good output

so the idea here is that

for example you're supposed

to leverage the fact that

you get multiple possible

plausible outputs and that

you can explore a design space

for example so this slide

amplifies this message

right so on the left side

the deep learning systems a

lot of the guidelines tend

to assume that the AI systems

produce a recommendation

or some action that the

users should either accept

or reject and that the AI

output, the one AI output

that comes out, is either

right or wrong or maybe it

might be imprecise to some

extent. So the guidelines

are around supporting efficient

correction and so forth.

Whereas the generative

AI guidelines are really

much more focused about on

the process of engagement

between the AI and the

human. So it's talking about

exploration of possibility

spaces, how users might be

supported in generating

multiple options, and then

Timestamp 44 minutes potentially explore and learn

and find inspiration in what

the right output should be.

So one of the guidelines that I really like was help user craft effective outcome specifications. So the idea here is that you're actually working with the system to understand what the output should be and how you want to evaluate it.

There's a short paper, which I haven't made as a reading that you have to do. It's not a requirement, but if you wanted to have a look at it, it's interesting. It's an extended abstract, so it's not really peer-reviewed paper, but someone has gone in and tried to extract the common patterns of how people are now building with LLMs, and the standard promptings, one, there's four or five different visualizations like this. It's a very short paper for five pages, so if you're interested, at least having a quick look at what people thought in January last year are the common patterns and practices is quite interesting, and it might have changed since, but it will still be quite interesting to have a look at.

Finally, here's a blog post which is required reading, and it's not awfully short, but it's also not awfully long, And what I'll try and do in the last couple of slides is to really pull out the key ideas that I want you to focus on. I won't give you all the content, but I'll just point you to the places which I think

are interesting and why
I'm sharing this blog post
as one of the readings.
So the whole idea is about
building with AI as agents.
And Anthropic kind of
differentiates between the
building blocks, workflows
and agents. And I think
that's what I'd really want
you to be able to talk about
and think about. What is
the difference between
these three things and how
would you use them in your
systems? And it's also
one of the things that you
will have to do as part
of the SGTs. We will be
building systems like
this. So the idea is that a
building block is just an
agent that might use, sorry,
Timestamp 46 minutes is like an LLM call that
might use tools and get some
sort of input and get some
sort of output. So that's
the core little Lego
break that you might have.
A workflow, there's multiple
versions of the workflow,
so do go into how a
workflow would look like, so
do go into the blog post,
but a workflow is a series
of predetermined paths
that the LLMs might take,
right? So it might be
that you have some input
coming in, you do a call,
you get an output, then you
have some traditional
system making a selection
whether the output is good
or bad, and if it's good,
then you might have a couple
more LLM calls, and then
some transformed data
comes up as the output. so
it's kind of very linear

and usability heuristics
as patterns of engaging
and solving design problems
and questioning whether your
system runs into any sort
of design errors in the pre-
gen era and those are really
well specified remember
how detailed the individual
guidelines have been and
so then what we started
doing towards the end and we
will kind of keep on coming
back to you throughout the
course isn't thinking well
well how does this traditional
model of designing for
usability or user experience
and so on apply to AI
systems which are much newer
and we have much kind of
less detailed frameworks
around both in terms of the
deep learning traditional
decision curation models, as
Timestamp 49 minutes well as the generative AI
models that are coming up.

This slide is here again
just from the first week to
indicate how by now you should
be able to quite cleanly
sort out the information
that you're getting and
understand how much of the
usability actually does
apply to generative AI systems
too. So the idea is that
all of those usability
goals will likely be still
valid for gen AI systems or
deep learning AI systems as
well. It's just that these
types of techniques have
some additional guidelines
on top because it's harder
to deploy or design these
tools with technology that
kind of quite literally
answers back in this case.
So this then takes us to

next week. And next week
you'll start thinking
about what does it mean to
understand requirements in
human AI systems so we'll
start talking about design
thinking and the individual
steps that you take in
this as well as start
understanding what does it
mean that people need something
how would we collect that
information how do we
design it and so forth so
I hope this has been useful
and I look forward to
seeing you in the LGT's bye